谛听 PdM · 实施工程（impl）

> 决策基线：见上级目录 `DECISIONS.md`（D1 – D25 已确认） 当前阶段：Ph1 垂直切片（可运行、可测试、可部署，尚未接真实设备）

1. 现在有什么

模块	路径	状态	说明
信号处理核心	`packages/pdm_signal/`	可用	时域/频域/包络/轴承特征频率/HI，纯函数，44 项测试
诊断服务	`services/python-diagnosis/`	可跑	Litestar + Granian + msgspec，`/v1/diagnose`，多租户守卫，多级缓存
数据模型	`db/tidb/001_schema.sql`	可用	TiDB 8.5 语法，tenant_id 打头，Outbox + 审计链
隔离规范	`db/tidb/TENANT_ISOLATION.md`	生效	五层防线 + TenantQueryBuilder 参考实现
跨语言契约	`contracts/proto/pdm.proto`	可用	资产/监测/诊断/预后/决策/事件
部署清单	`deploy/k8s/pdm-prod.yaml`	可 apply	命名空间/配额/网络策略/Deployment/Ingress
CI 门禁	`.github/workflows/ci.yml`	可用	测试+覆盖率+SAST+密钥扫描+清单校验
压测工具	`tools/bench.py`	可用	吞吐/P50/P95/P99，用于性能门禁
文档导出	`tools/md_export.py`	可用	全部 Markdown → Word + PDF（30 源文件 → 60 文件）

尚未实现（按 Ph1 – Ph4 排期）：Rust 采集与 DSP、Go 事件管线与 GraphQL BFF、Java 资产/知识/集成、TDengine 写入、Pulsar 投递、决策会诊、方案卡、采购护栏。

1.1 技术栈（性能优先）

层	选型	相对原方案
Web 框架	Litestar 2.25	替代 FastAPI，链路更薄、msgspec 原生
ASGI 服务器	Granian 2.x（Rust/Hyper）	替代 uvicorn，实测 +21.5% 吞吐
序列化	msgspec 0.22	替代 pydantic，Rust 侧编解码
缓存	L1 TTL(30s) + L2 Redis(120s)	新增，压测命中率 99.94%

实测（Windows 开发机，2 workers，1500 请求 / 并发 40，缓存命中路径）：

服务器	吞吐	P95
Granian	717 req/s	158 ms

服务器	吞吐	P95
uvicorn	590 req/s	185 ms

> 数据仅用于证明选型方向，生产指标以 `docs/08` 的性能测试为准。详见 `docs/19`。

2. 5 分钟跑起来

```
cd impl

# 1) ■■
python -m venv .venv
.venv/Scripts/python.exe -m pip install -r services/python-diagnosis/requirements.txt   # Windows
# .venv/bin/python -m pip install -r services/python-diagnosis/requirements.txt         # Linux/macOS

# 2) ■■■■■■■■■■
make test

# 3) ■■
make run
# ■■■■■■ http://127.0.0.1:8000/docs
```

冒烟请求：

```
curl -X POST http://127.0.0.1:8000/v1/diagnose \
  -H "Content-Type: application/json" \
  -H "X-Tenant-Id: site-a" \
  -d '{
    "tenant_id": "site-a",
    "asset_id": "KA-101",
    "metrics": {
      "vib_rms_mm_s": 4.0,
      "vib_baseline_mm_s": 2.0,
      "kurtosis": 6.5,
      "temp_c": 68,
      "temp_baseline_c": 55,
      "trend_pct_per_day": 1.2
    }
  }'
```

实测返回（已验证）：

```
{
  "hi": 54.94,
  "level": "orange",
  "sla_hours": 72,
  "degraded": true,
  "degraded_reason": "■■■■■: curr■■■■■■■■■■■■■■■■■■",
  "contributions": [
    {"name": "vib",  "s": 0.5,   "impact": 20.59, "evidence": "■■RMS 4.0 mm/s■■■■■■ 2.0 mm/s■"},
    {"name": "imp",  "s": 0.625, "impact": 14.71, "evidence": "■■■ 6.5■>4 ■■■■"},
    {"name": "temp", "s": 0.867, "impact": 15.29, "evidence": "■■■ 13.0■■15■ ■■■■■■"},
    {"name": "trend","s": 0.24,  "impact": 4.24,  "evidence": "■■■■■ 1.2%/■■5%/■ ■■■■■■"}
  ],
  "disclaimer": "■■■■■ AI ■■■■■……■■■■■■■……"
}
```

> 注意 `degraded: true`：只给了 4 个维度，缺 `curr`。系统不会假装完整，这是 P3 原则的落地。

3. 关键设计决策（为什么这么写）

决策	代码位置	理由
HI 用乘积不用加权和	`health.py:compute_hi`	单维度致命必须归零；乘积避免多维度重复扣分
维度缺失要降级，不静默补零	`health.py` + `main.py`	缺数据说"缺数据"，不能说"正常"

决策	代码位置	理由
轴承诊断要求 ≥2 阶谐波 + SNR≥3	`bearing.py:is_suspicious`	单峰是噪声，不是故障；这是误报率的第一道闸
用中位数/MAD 而非均值/标准差	`signal.py:robust_baseline`	工业数据非高斯、有离群点
租户 header 与 body 必须一致	`main.py:diagnose`	防参数篡改；这是 L2 防线
免责声明不可关闭	`main.py:DISCLAIMER`	合规（GB/T 45654）+ 责任边界
证据与结论同返回	`main.py:evidence`	无证据的结论不得输出（宪法第三条）

---

4. 已知限制（不要误以为能上线）

#	限制	影响	解除条件
L1	无数据库读写	每次请求独立计算，不落库	接入 TiDB（需 I2 账号）
L2	无 TDengine 写入	时序数据只能靠请求体传入	部署 TDengine + 采集链路
L3	无 Pulsar 投递	不产生领域事件，下游收不到	Pulsar tenant 权限（I1）
L4	轴承几何需人工填	无法自动从台账取	接入资产上下文（Java）
L5	未做鉴权	只校验 header 一致性，未验签	接 OIDC
L6	单语言	Rust/Go/Java 尚未实现	按 Ph1 – Ph3 排期

当前是"可验证的算法内核"，不是"可交付的系统"。

---

5. 下一步（Ph1 剩余工作）

按依赖顺序：

- TiDB 落地：建库 → 灌 DDL → 接 `TenantQueryBuilder` → 诊断结果落库（阻塞：I2 账号）
- TDengine 落地：部署 → 特征值写入 → 查询 API（阻塞：I4 磁盘）
- Pulsar 打通：建 `pdm` tenant → Outbox relay → 事件可观测（阻塞：I1 权限）
- Rust 采集骨架：OPC UA Client 读 1 台设备 → 特征计算 → 投递
- Go 事件管线：消费 `diagnosis.finalized` → 投影到读模型
- Java 资产服务：组织树 + 设备台账 + 权限（D1 要求总部+2 基地）

---

6. 工程纪律

- 提交前 `make test` 必须全绿；覆盖率不得低于 80%
- 任何新增数据访问必须经 `TenantQueryBuilder`，禁止裸 SQL
- 任何新增判据必须先在 `pdm_signal/tests/` 写用例
- 契约（proto）只增不改；破坏性变更升版本并双跑
- 不许删改 `TENANT_ISOLATION.md §5` 的四条越权用例